Engenharia de Software I

Prof. Dr. Manuel Gonçalves da Silva Neto
manuel@ifpi.edu.br

Agenda:

- Introdução ao Desenvolvimento Ágil (de forma geral)
- Apresentação do Extreme Programming (XP)

Sumário

- Deficiência dos modelos de processo prescritivos
- Origem do Desenvolvimento Ágil
- Filosofia do Desenvolvimento Ágil
- Características do processo ágil
- Fatores humanos no Desenvolvimento Ágil
- Programação Extrema – XP
- Outros modelos de processos ágeis

Métodos Ágeis – História

- No início da década de 90 havia uma visão de que a melhor maneira para se criar software era por meio de um **planejamento cuidadoso,** formalidades e projeto apoiado por CASE;
- Essa visão veio da comunidade da Engenharia de Software, responsável por sistemas de softwares grandes, como sistemas aeroespaciais e do governo.

Métodos Ágeis - História

- O *overhead* (sobrecarga) de planejamento estava se tornando tão pesado que estava dominando o processo de desenvolvimento. Gastava-se mais tempo em análises do que na construção do programa e testes.
- A insatisfação levou ao surgimento, no final da década de 90, dos novos '***métodos ágeis***'.
- "*Estamos descobrindo melhores maneiras de desenvolver softwares, fazendo-o e ajudando outros a fazê-lo*"...

Deficiência dos modelos de processo prescritivos

- Requerem muita disciplina para o sucesso em seu uso;
- Entretanto, engenheiros de software podem não ter toda a disciplina necessária para tal;
- Além disso, vivemos em um mundo **repleto de mudanças (escopo, tempo e custo)**:
  - No software em desenvolvimento;
  - Na equipe;
  - Nas tecnologias utilizadas.
- E modelos prescritivos apresentam maior dificuldade em responder a tais mudanças.

Origem do Desenvolvimento Ágil

- Manifesto para o Desenvolvimento Ágil de software (2001), que valoriza:
  - **Indivíduos e interações** **acima** de processos e ferramentas;
  - **Software operacional** **acima** de documentação completa;
  - **Colaboração dos clientes** **acima** de negociação contratual;
  - **Respostas a mudanças** **acima** de seguir um plano.
- Embora haja valor no que está relacionado à direita, prioriza-se o que é exposto à esquerda!
- Por ágil, deve-se entender "*capaz de responder de forma rápida e apropriada a mudanças*"!

Métodos Ágeis – Mais conhecidos

- XP – Extreme Programming
- SCRUM
- Crystal
- DSDM
- FDD

Métodos Ágeis – Princípios

- Envolvimento do cliente
- Entrega Incremental
- Pessoas, não processos
- Aceitar as mudanças
- Manter a simplicidade

Filosofia/princípios do Desenvolvimento Ágil

- Satisfação do cliente por meio de **entregas incrementais**;
- Equipes de projetos **pequenas e bem motivadas**;
- **Simplicidade** no desenvolvimento **reduzindo a quantidade de artefatos** de Engenharia de Software produzidos;
- **Comunicação contínua** entre todos os envolvidos;
- Emprega as mesmas atividades metodológicas no processo, porém com **foco na entrega**.

Custo da mudança em função do tempo

Características de um processo ágil

- **Adaptável** – equipe de desenvolvimento pode adaptá-lo segundo suas necessidades;
- **Incremental** – entrega de versões incrementais do software, importante para validação junto ao cliente;
- **Iterativo** – desenvolvimento se dá por meio de iterações, assim as primeiras iterações não precisam lidar com toda a complexidade do software;
- **Focado na comunicação** – maior foco na comunicação contínua, em vez de ter foco na execução segundo um contrato ou plano rígido.

Métodos Ágeis – Indicação

- Desenvolvimento de produtos pequenos ou médios;
- Sistemas personalizados dentro de uma organização , em que existe um **compromisso claro do cliente de se envolver**, com poucas regras ou regulamentos externos que afetam o software.

Métodos Ágeis – Dificuldades

- Sucesso depende de um cliente disposto e capaz de passar o tempo com a equipe de desenvolvimento;
- Alguns membros podem ter dificuldades em interagir com outros da equipe;
- Priorizar mudanças é difícil quando existem muitos *stakeholders;*
- Manter a simplicidade exige trabalho extra;
- Migrar de processos bem definidos para um processo mais informal pode ser difícil.

Métodos Ágeis – Quando usar

Na verdade, a maioria dos projetos de software inclui práticas das abordagens dirigidas a planos e ágeis:

- **Se é importante ter ou manter uma especificação e vasta documentação:** usar abordagem dirigida a planos;
- **Se o feedback do cliente for rápido:** métodos ágeis;

O uso de métodos ágeis demanda/necessita de boas ferramentas, bons projetistas e programadores.

Programação Extrema – XP (Extreme Programming)

- Uma abordagem para desenvolvimento ágil;
- Criado em 2000 por *Kent Beck*, em XP os requisitos são expressos por **estórias de usuário**. Quando o código novo é integrado ao sistema, todos os testes devem ser executados com sucesso.
- "Ajusta-se bem a equipes pequenas e médias em desenvolvimento de software com requisitos vagos e em constante mudança. Para isso, adota a estratégia de constante acompanhamento e realização de vários pequenos ajustes durante o desenvolvimento de software."

XP – Extreme Programming

Fundamentos:

- O desenvolvimento incremental é sustentado por meio de pequenos e frequentes releases;
- Engajamento contínuo do cliente;
- Pessoas e não processos sustentado pela programação em pares;
- Mudanças são aceitas por meios de *releases* contínuos para clientes, *test-first* e integração contínua;
- Simplicidade é mantida por meio da **refatoração** constante.

XP – Extreme Programming

Práticas:

- Planejamento incremental;
- Pequenos *releases*;
- Projeto simples;
- *Test-first*;
- Refatoração;
- Programação em pares;
- Propriedade coletiva;
- Integração contínua;
- Ritmo sustentável;
- Cliente no local;

Valores da XP

1. **Comunicação**, por meio de:
   - Comunicação estreita e informal entre clientes e desenvolvedores;
   - Metáforas (**estória de usuário** – uma descrição sobre como o sistema funciona) para comunicar conceitos importantes;
   - *Feedback* contínuo;
   - Evitar excesso de documentação na comunicação.

Valores da XP

2. **Simplicidade**, por meio de:
   - Foco apenas nas necessidades imediatas, ignorando as futuras;
   - Se necessitar de melhorias, o projeto pode ser refatorado (retrabalhado);
   - Refatoração (do inglês *Refactoring*) é o processo de modificar um sistema de software para melhorar a estrutura interna do código sem alterar seu comportamento externo.

Valores da XP

3. **Feedback**, por meio:
   - Do próprio software implementado, a partir de resultados de testes;
   - Do cliente, a partir de testes de aceitação considerando as histórias de usuário ou casos de uso;
   - De outros membros da equipe, referente ao impacto nos custos e no cronograma que certas mudanças podem causar.

Valores da XP

4. **Coragem / disciplina**, a fim de “projetar para hoje”, uma vez que necessidades futuras podem sofrer alterações drásticas em qualquer momento, levando a retrabalho;

4. **Respeito**, conforme as versões incrementais do software são entregues com sucesso.

Processo XP

- Adota principalmente a Orientação a Objetos;
- Envolve quatro atividades metodológicas:
  - **Planejamento;**
  - **Projeto;**
  - **Codificação;**
  - **Testes.**

Processo XP

1. **Planejamento**
   - Inicia com a elicitação (levantamento) de requisitos, a fim de compreender o ambiente de negócios em que o *software* executará e funcionalidades esperadas;
   - Conduz o desenvolvimento de *user stories* (estórias de usuários) descrevendo cada funcionalidade requisitada;
   - Para cada *user story*, o cliente atribui um valor (prioridade) e a equipe de desenvolvimento, um custo (tempo para implementar aquela funcionalidade);
     - Caso uma estória demore mais que três semanas, o cliente deve dividi-la em estórias menores.

User story (Estória de usuário)

- Um dos artefatos primários no desenvolvimento ágil. Amplamente usado em *Scrum* e XP;
- É uma definição de alto nível de um requisito, contendo apenas informação suficiente para que desenvolvedores possam definir uma estimativa do esforço (tempo) para implementá-la;
- É escrita em cartão de 8 cm x 13 cm.

XP – Extreme Programming

Estória de usuário (exemplo simples):

Kate é uma médica que deseja prescrever medicamentos para um paciente de uma clínica. O prontuário do paciente já está sendo exibido em seu computador, assim, ela clica no campo “medicação” e pode selecionar “medicação atual”, “nova medicação” ou “formulário”.Se ela selecionar “medicação atual”, o sistema pede que ela verifique a dose.Se ela quiser mudar a dose, ela altera esta e em seguida, confirma a prescrição. Se ela escolher “nova medicação”, o sistema assume que ela sabe qual medicação receitar. Ela digita as primeiras letras do nome do medicamento. O sistema exibe uma lista de possíveis fármacos que começam com essas letras. Ela escolhe a medicação requerida e o sistema responde, pedindo-lhe para verificar se o medicamento selecionado está correto. Ela insere a dose e, em seguida, confirma a prescrição.

User story (Estória de usuário)

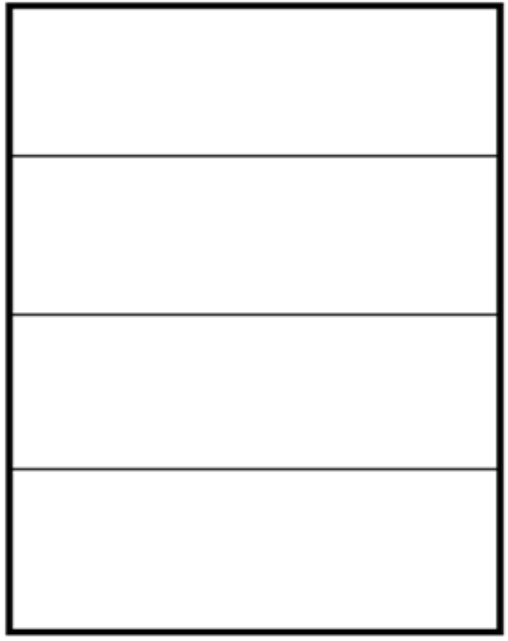
- Após refinada ela pode assumir o seguinte formato:

  Como um(a) <papel>, [quando <evento>] eu quero <ação> [tal que <resultado esperado>].

- Exemplos:
  - Como um administrador, eu quero poder cadastrar novos funcionários;
  - Como um leitor, quando eu atrasar a devolução de um livro, eu quero receber um e-mail notificando isso.

Vamos praticar!

Agora, elabore 1 estória de usuário para pelo menos um dos seguintes sistemas: biblioteca, controle acadêmico, jogo de tiro por equipes e Sistema do restaurante universitário.



Exemplos refinados:

- ✓ Como um administrador, eu quero cadastrar novos funcionários;
- ✓ Como um leitor, quando eu atrasar a devolução de um livro, eu quero receber um e-mail notificando isso.

Processo XP

2. **Projeto**
   - Segue o princípio KISS (*keep it simple, stupid*);
   - Oferece um guia para implementação das *user stories* na medida em que são escritas;
   - Encoraja-se o uso de cartões CRC (classe-responsabilidade-colaborador);
   - Desenvolvimento de soluções pontuais:
     - Caso um difícil problema de projeto (a partir de uma *user story*) seja encontrado, recomenda-se implementar e avaliar um protótipo.

XP – Extreme Programming

- Os cartões são as principais entradas para o processo de planejamento no XP ou o “jogo de planejamento” (*planning game*).
- Uma vez que tenham sido desenvolvidos, e equipe os divide em tarefas e estima o esforço e os recursos necessários para a realização de cada tarefa.
- O Cliente é quem prioriza as estórias para implementação. Assim como os requisitos mudam, as estórias podem mudar ou até mesmo serem descartadas.

Exemplo de divisão da estória em tarefas:

- Tarefa 1: Alterar dose de medicamentos prescritos
- Tarefa 2: Seleção de formulário
- Tarefa 3: Verificação de dose
- ……..

Cartão CRC

- O modelo CRC é uma coleção de cartões (8 cm x 13 cm) divididos em três seções (Classe, Responsabilidades e Colaboradores);
  - Uma **classe** é uma coleção de objetos similares (ex: coleção de livros, usuários, empréstimos etc.);
  - Uma **responsabilidade** é algo que uma classe sabe ou faz;
  - Um **colaborador** é outra classe com quem nossa classe interage a fim de cumprir suas responsabilidades.

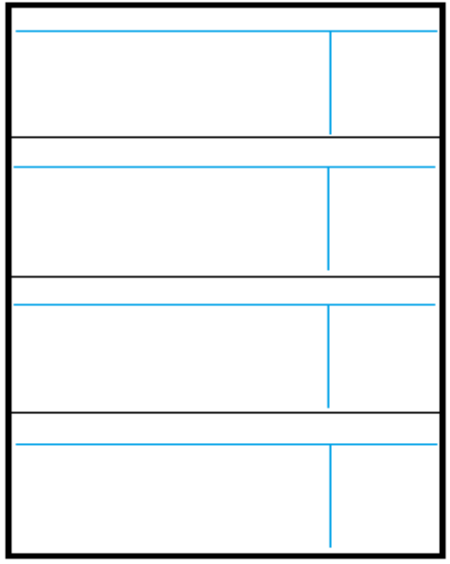
Cartão CRC

- Exemplo de Cartão CRC:

Classe Leitor	
Responsabilidades Faz empréstimo Sabe nome Sabe endereço Sabe telefone Sabe histórico de empréstimos	**Colaboradores** Empréstimo

Vamos praticar!

Agora, elabore 4 cartões CRC para um sistema de biblioteca!



Classe Consumidor	
Responsabilidades Faz pedidos Sabe nome Sabe endereço Sabe telefone Sabe histórico de pedidos	**Colaboradores** Pedido

Processo XP

3. **Codificação**

- Um pouco mais sobre **programação em pares**, outra técnica inovadora do XP, consiste em dois programadores escreverem juntos uma mesma funcionalidade, lado a lado, na mesma estação de trabalho. Os pares são criados de forma dinâmica, de modo que todos os membros da equipe trabalhem uns com os outros.
- Vantagens da programação em pares:
  - Dá suporte à idria de propriedade e responsabilidade coletiva para o sistema;
  - Atua como um processo de revisão informal, pois cada linha de código é observada por, pelo menos, duas pessoas.
  - Dá suporte à refatoração, que é um processo de melhoria de software.

Processo XP

4. **Testes**
   - Testes de unidade são definidos antes da codificação;
   - Foco na automatização dos testes de unidade;
   - Encorajamento de testes de regressão;
   - Testes de aceitação (testes de cliente) são obtidos das estórias de usuário e possuem como foco as características do sistema que o cliente pode revisar.

Exemplo de teste de unidade automatizado

```
public class MathExt {
  public static void factorial(int x) {
      if (x < 0) {
                return 0;
          } else if (x == 0) {
                return 1;
          } else {
                return x * factorial(x-1);
          }
  }
}
```

Classe a ser testada

```
public class Tester {
  public static void main(String[] args) {
          if (MathExt.factorial(-1) != 0) {
                Logger.log("MathExt.factorial(-1) failed!");
          }
          if (MathExt.factorial(0) != 1) {
                Logger.log("MathExt.factorial(0) failed!");
          }
          if (MathExt.factorial(5) != 120) {
                Logger.log("MathExt.factorial(5) failed!");
          }
          Logger.log("Test is finished.");
  }
}
```

Classe testadora

Processo XP

projeto simples
cartões CRC
soluções pontuais
protótipos
valores das histórias
de usuários
critérios de teste de aceitação
plano de iteração
projeto
planejamento
codificação
refabricação
teste
programação em dupla
Versão
incremento de software
velocidade de projeto registrada
(computada)
teste de unidades
integração contínua
teste de aceitação

Críticas à abordagem XP

- **Volatilidade de requisitos**. Como o cliente é um membro ativo na XP, alterações podem ser solicitadas informalmente, levando a possíveis mudanças de escopo;
- **Necessidades conflitantes de clientes**. Em XP, a equipe de desenvolvimento deve assimilar as necessidades de múltiplos clientes, muitas vezes conflitantes;

Críticas à abordagem XP

- **Requisitos elicitados informalmente**. As estórias de usuários são a única manifestação explícita dos requisitos, não havendo assim um modelo mais formal para evitar omissões e inconsistências;
- **Falta de projeto formal**. Sistemas complexos podem requerer maior elaboração do projeto para garantir qualidade e facilidade de manutenção do software, mas a XP não enfatiza a necessidade do projeto da arquitetura.

Fatores humanos no Desenvolvimento Ágil

- O processo deve adequar-se às necessidades da equipe de desenvolvimento e não o contrário!
- Membros de uma equipe ágil devem ter:
  - Competência (talento inato + habilidades relacionadas a desenvolvimento de software + conhecimento generalizado do processo escolhido);
  - Foco comum (entregar um incremento de software funcionando ao cliente, no prazo definido);
  - Habilidades de colaboração, tomada de decisão, solução de problemas e auto-organização;
  - Confiança mútua e respeito.

Problemas com métodos ágeis

- Pode ser difícil manter o interesse dos clientes que estão envolvidos no processo.
- Membros da equipe podem não ser adequados ao envolvimento intenso que caracteriza os métodos ágeis.
- Priorizar mudanças pode ser difícil onde existem múltiplos *stakeholders*.
- Manter a simplicidade requer trabalho extra.
- Os contratos podem ser um problema assim como em outras abordagens que usam o desenvolvimento iterativo.

Outros modelos de processos ágeis

1. Scrum;
2. Desenvolvimento de Software Adaptativo;
3. Método de Desenvolvimento de Sistemas Dinâmicos;
4. Crystal;
5. Desenvolvimento Dirigido a Funcionalidades (FDD);
6. Desenvolvimento de Software Enxuto;
7. Modelagem Ágil;
8. Processo Unificado Ágil (Agile UP).

Exercitando

Complete

1. Por ágil, deve-se entender “ser capaz de _____ de forma rápida e apropriada a _____ ” .

Múltipla Escolha

2. Marque com um X as opções que fazem parte da filosofia do desenvolvimento ágil:

a ( ) Obediência à execução sequencial das atividades metodológicas;

b ( ) Satisfação do cliente por meio de entregas incrementais;

c ( ) Equipes de projetos pequenas e bem motivadas;

d ( ) Eliminação de toda a documentação;

e ( ) Comunicação contínua entre todos os envolvidos.

Complete

3. Modelos de processo _____ requerem muita disciplina, entretanto engenheiros de software podem não ter toda a disciplina necessária.

Já os modelos de processo _____ não requerem tanta disciplina e visam responder apropriadamente às mudanças.

Complete

4. No desenvolvimento ágil, devido à importância da entrega incremental do software, é comum o emprego de um fluxo de processo _____ .

Verdadeiro ou Falso

5. ( ) O processo XP envolve quatro atividades: planejamento, projeto, codificação e testes.

Múltipla Escolha

6. Marque com um X os itens **que contêm valores** da abordagem XP:

a) ( ) Comunicação;

b) ( ) Planejamento;

c) ( ) Simplicidade;

d) ( ) Feedback;

e) ( ) Documentação;

f) ( ) Coragem / Disciplina;

g) ( ) Respeito.

Verdadeiro ou Falso

7. ( ) Em XP, desenvolvem-se *user stories* com o intuito de descrever cada funcionalidade requisitada.

Verdadeiro ou Falso

8. ( ) Em XP, inicia-se a atividade de planejamento com o desenvolvimento de uma série de testes de unidade para cada história de usuário.

Verdadeiro ou Falso

9. ( ) Não se recomenda a programação em pares no processo XP, pois alocar dois programadores para implementar uma mesma funcionalidade seria um grande desperdício.

Complete

10. Em XP, caso um difícil problema de projeto seja encontrado, recomenda-se implementar e validar um _____ .

Responda

11. Em nossas aulas, comentamos que “vivemos em um mundo repleto de mudanças”. Em se tratando de desenvolvimento de software:

a) Quais são essas possíveis mudanças?

b) Quais as consequências delas para o desenvolvimento de um software?

c) O que pode ser feito para minimizar o impacto de tais mudanças?

Responda

12. Explique com suas palavras o que o Manifesto Ágil quer dizer quando afirma que deve se priorizar:

a) Indivíduos e interações acima de processos e ferramentas?

b) Software operacional acima de documentação completa?

c) Colaboração dos clientes acima de negociação contratual?

d) Respostas a mudanças acima de seguir um plano?

Responda

13. Explique o que significa cada uma das seguintes características do desenvolvimento ágil:

a) Adaptável;

b) Incremental;

c) Iterativo;

d) Focado na comunicação.

Responda

14. Descreva agilidade (no contexto de desenvolvimento de software) com suas palavras;

Responda

15. Por que o processo iterativo facilita o gerenciamento de mudanças?

Responda

16. O que é refatoração?

Responda

17. Cite pelo menos uma vantagem da programação em duplas (pares)?

Escreva

18. Escolha um dos modelos de processo ágil citados (mas não descritos), exceto o Scrum, pesquise na web ou em livros de engenharia de software e escreva a respeito de seus fundamentos , atividades, processo e visão geral de forma bem resumida. Ao final, tente fazer um comparativo entre o seu modelo escolhido e o XP.

See you soon!